ScorpioSync-Kaleidoscorp Ω

A Pure-Rust Runtime Master Specification

BCIK-Gated Projective Connection Hypercube — Identity/Locator Separation, Boundary-Closed Stitch Fabrics, and Reanalysis-Certified Connection Events

This document translates the SSK Ω formal framework into a closed, implementable, testable and auditable **pure-Rust runtime specification**. It is written so that a Rust coding agent can generate a working multi-crate repository from it: it fixes types, traits, modules, state machines, gate contracts, serialization formats, error taxonomies, test protocols, feature flags, a repository layout, and staged conformance levels. The document preserves the architecture of SSK Ω and preserves its claim discipline: it makes no claim of untraceability, no covert-channel instruction, no self-designed cryptography, and no absolute anonymity. Privacy is modeled strictly as *controlled separation with bounded, measurable linkability and visible residues.*

Sebastian Klemm

Autonomous technical evolution synthesis — next implementation layer

3 July 2026

Contents

1 Purpose, Scope, and Design Boundary **4**
- 1.1 Position of this document . . . 4
- 1.2 What the runtime is . . . 4
- 1.3 What the runtime is not (normative non-goals) . . . 4
- 1.4 Model separation . . . 5

2 Architectural Extraction and Term Normalization **6**
- 2.1 Extraction table . . . 6
- 2.2 Claim discipline carried into code . . . 7

3 Rust Type Discipline and Identifiers **8**
- 3.1 Newtype identifiers and content addressing . . . 8
- 3.2 Sealed traits for foundation invariants . . . 8
- 3.3 Typestate for the connection lifecycle . . . 9
- 3.4 Secrets and zeroization . . . 9
- 3.5 Error and result conventions . . . 10

4 Core Domain Model **11**
- 4.1 Core node and mirror facets . . . 11
- 4.2 Birefringent split: identity ray and locator ray . . . 12
- 4.3 Locator facets . . . 12
- 4.4 Protocol fibers . . . 13

5 Cryptographic and Capability Layer **14**
- 5.1 Provider traits . . . 14
- 5.2 Capability witness . . . 15

6 Locator and Relay Model **16**
- 6.1 Locator classes and metadata . . . 16
- 6.2 Locator pool and adapter interface . . . 16
- 6.3 Explicit stitch edges and the funnel-DAG . . . 16
- 6.4 Path selection . . . 17

7 Hypertoroidal Scheduler **18**
- 7.1 Phase representation . . . 18
- 7.2 Deterministic and seeded modes . . . 18

8 Projective Connection Hypercube **20**
- 8.1 Axes . . . 20
- 8.2 Projection check . . . 20

9 BCIK and Boundary Model **22**
- 9.1 Crossing kernel . . . 22
- 9.2 Residue taxonomy . . . 23

10 Gabriel Constraint Cells **24**

11 Gate System and Gate Vector **25**
11.1 Gate contract . . . 25
11.2 The twelve gates . . . 26
11.3 Gate profile . . . 27

12 Persistent Stitch Tensor **28**
12.1 Structure . . . 28
12.2 Update semantics . . . 28
12.3 Storage backends . . . 29

13 Concave Reanalysis Focus **30**
13.1 Types . . . 30

14 Runtime Semantics and State Machine **31**
14.1 Pipeline . . . 31
14.2 Legitimate outcomes . . . 31

15 Threat and Privacy Model **32**
15.1 Observable surfaces . . . 32
15.2 Adversary classes and honest capability matrix . . . 32
15.3 Linkability as a metric . . . 33

16 Data, Audit, and Replay Model **34**
16.1 Canonical encoding . . . 34
16.2 Persisted record types . . . 34
16.3 Replay engine . . . 35

17 Faultscan **36**
17.1 Fault taxonomy . . . 36
17.2 Fault matrix . . . 37

18 Validation and Testing **38**
18.1 Test tiers . . . 38
18.2 Deterministic test vectors . . . 38
18.3 Property obligations . . . 38
18.4 Privacy metric tests . . . 39
18.5 Defensive adversarial simulation . . . 39
18.6 Conformance test map . . . 40

19 Pure-Rust Implementation Blueprint **41**
19.1 Hard constraints . . . 41
19.2 Feature flags . . . 41
19.3 Recommended patterns . . . 41
19.4 Dependency posture . . . 42

20 Repository Blueprint **43**

21 Configuration Model **44**

22 Conformance Levels **45**

23 Implementation Phases and Minimal Viable Runtime **46**
23.1 Minimal Viable Runtime (MVR) . . . 46
23.2 Phase plan . . . 46

24 Implementation Contract and Stated Limits **47**
24.1 Conformance contract 47
24.2 What the system does and does not claim 47

1 Purpose, Scope, and Design Boundary

1.1 Position of this document

ScorpioSync-Kaleidoscorp Ω (**SSK Ω**) remains the system identity and the target body. The source framework document defines the *formal* object grammar of the runtime. This document is the *implementation-normalized* successor layer: it does not replace the framework, and it does not re-open its claims. It fixes exactly how the framework becomes a running, testable, auditable Rust artifact.

The reader of this document is assumed to be a Rust engineer or a Rust coding agent. Every central object of SSK Ω is therefore given a concrete Rust surface: a type, a trait, an error class, a state transition, a serialization contract, and a test obligation. Where the framework is deliberately abstract, this document marks the seam between *fixed* and *pluggable* rather than inventing detail that would not survive contact with a real network stack.

1.2 What the runtime is

SSK Ω is a privacy-aware connection runtime. A connection event is not a direct identification of a client core by its public network locator. It is a controlled crossing between two typed planes:

$$\mathsf{ConnectionEvent} = \mathsf{IdentityPlane} \oplus \mathsf{LocatorPlane} \oplus \mathsf{StitchClosure}.$$

The identity side carries the canonical core, mirror facets, session intent, key context, policy and replay root. The locator side carries direct IP, VPN exit, proxy, relay, gateway, port, route, timing and observable transport surface. The crossing is mediated by a boundary-closed stitch fabric, evaluated by a gate vector, materialized into a persistent stitch tensor, and reconstructed by concave reanalysis focus.

1.3 What the runtime is not (normative non-goals)

The following non-goals are normative. A conforming implementation *must not* claim or advertise any of them, and *must not* contain features whose sole purpose is to achieve them.

- **Non-goal:** No magic address creation. Mirror facets never synthesize public IP addresses. External locator plurality exists only when real egress resources exist.
- **Non-goal:** No untraceability and no absolute anonymity. The runtime reduces *bounded, declared* linkability under a declared adversary model; it does not defeat network reality.
- **Non-goal:** No covert-channel, no traffic-obfuscation-for-evasion, and no censorship-circumvention tooling. A "locator" is a typed egress/transport resource, not an evasion device. The specification defines resource *typing, accounting and audit*, never operational concealment procedures.
- **Non-goal:** No self-designed cryptography. The runtime consumes cryptographic primitives only through abstract provider traits backed by established, audited implementations.
- **Non-goal:** No silent loss. Every unresolved component of a crossing becomes a typed, visible residue. There is no "drop and forget" path.

Axiom 1.1 — Locator/Identity separation of concern

An IP address, VPN exit, proxy endpoint, relay or gateway is a *locator facet*. It is never the canonical core identity. A relation between an identity facet and a locator facet is valid only as an explicit stitch event carrying type, lifetime, capability witness, boundary record, gate report, trace and replay handle.

Axiom 1.2 — Infrastructure honesty

The runtime can separate core identity from locator appearance only to the degree that real locator alternatives are configured. With a single direct public IP and no other egress resource, the runtime still provides internal session separation, gate discipline, capability scoping and audit, but it provides only weak network-level privacy, and it must report this as a high-linkability baseline condition rather than concealing it.

1.4 Model separation

The specification is factored into five models that map onto distinct Rust surfaces. This factoring is load-bearing for the repository layout in Chapter 20.

Model	Concern	Rust surface
Domain model	Timeless identities, facets, locators, stitches, gates	Pure `struct`/`enum`/`trait`; `no_std`-friendly core; no I/O
Runtime model	Scheduling, gate evaluation, tensor mutation, focus	Sync deterministic core, optional async edges; owns state machines
Persistence model	Append-only audit, tensor storage, replay manifests	`serde` data types + pluggable `StorageBackend`
Config model	Policy bundles, feature toggles, thresholds	`serde`-deserialized `Policy`; validated at load
Test model	Deterministic vectors, property tests, simulation	`proptest` strategies, sim harness, replay diff tooling

Layer order (fidelity constraint)

$$\text{Foundation} \prec \text{Runtime} \prec \text{Product} \prec \text{Domain} \prec \text{Audit/Residue}.$$

A higher layer may use the foundation but may not redefine it. A locator pool, VPN interface, proxy adapter or relay implementation instantiates a typed locator role; it can never redefine the identity model or bypass a gate. In Rust terms, foundation invariants are protected by *sealed traits* and private constructors, so downstream crates cannot forge core objects.

2 Architectural Extraction and Term Normalization

This chapter performs the first work pass: it extracts the load-bearing system objects from SSK Ω, separates *theory terms* from *implementation terms*, and neutralizes anything that is not implementable. The rule is strict: a symbolic or legacy name has no authority in the Rust runtime unless it maps to a typed object with inputs, outputs, gates, trace and residue policy.

2.1 Extraction table

Framework object	**Nature**	**Rust normalization**
CoreNode	Structural	`CoreNode` struct with canonical form + signature; private ctor
Mirror facet / orbit	Structural	`MirrorFacet` + `MirrorOrbit`; deterministic derivation from core
Identity/Locator birefringence	Process	`birefract()` function producing `IdentityRay`, `LocatorRay`
Locator plane / egress	Structural	`LocatorFacet` enum over egress classes; `LocatorPool`
BCIK (boundary-closed kernel)	Structural + process	`bcik` crate: `SeamCrossing`, `ResidueRegister`, `BoundaryOperator`
Projective Connection Hypercube	Structural	`Hypercube` axis projection engine; pure predicate checks
Hypertorus T^n	Runtime	`PhaseVector`/`FrequencyVector` scheduler; integer ticks
Gabriel constraint cells	Runtime	`ConstraintCell` trait; local advisory evaluators (no persist rights)
Capability witness layer	Crypto-adjacent	`CapabilityWitness` + `CapabilityWitnessProvider` trait
Funnel-DAG / stitch edges	Structural	`LocatorGraph` (DAG) with explicit typed `StitchEdge`
Persistent Stitch Tensor	Persistence	`StitchTensor` append-structured store; hash-chained audit
Gate vector	Runtime	`GateVector`: ordered `Gate` trait objects; fail-closed
Concave reanalysis focus	Runtime	`FocusEngine`: response grouping + canonical-class reconstruction
Audit / trace / replay	Persistence	`TraceRecord`, `ReplayManifest`, `RunDescriptor` (serde)
Dissolution / TTL	Runtime	`DissolutionManager`; epoch/TTL sweep

Framework object	Nature	Rust normalization
Faultscan	Runtime	`Fault` taxonomy enum + `Faultscan` detector; severity policy
Wormhole, brane, mandorla, ghost, resonance, kaleidoscope	*Metaphor*	**Neutralized.** No runtime object. Retained only as doc-level aliases mapping onto the typed objects above.
"Magic address", "perfect anonymity", "untraceable"	*Non-claim*	**Removed.** Forbidden as feature or advertised property (Chapter 1 non-goals).

2.2 Claim discipline carried into code

The framework accepts five normative sentence types (Definition, Axiom, Invariant, Process contract, Register decision). This document keeps that discipline and binds each to a Rust enforcement mechanism.

Sentence type	Meaning	Rust enforcement
Definition	Declares a type/operator	`struct`/`enum`/`trait` declaration
Axiom	Non-negotiable structural condition	Type invariant + private constructor + `debug_assert!`
Invariant	Preserved across transforms	Property test (`proptest`) + runtime check with residue on violation
Process contract	Sequence, gates, trace, replay	Typestate state machine; ordered gate vector
Register decision	Assign to core/adapter/residue	Module boundary + `Residue` emission path

Object card (mandatory)

Every runtime object carries an *object card*:

$$\mathsf{ObjCard} = (\mathsf{Name}, \mathsf{Type}, \mathsf{Layer}, \mathsf{Inputs}, \mathsf{Outputs}, \mathsf{Operators}, \mathsf{Gates}, \mathsf{Trace}, \mathsf{Replay}, \mathsf{Residues}).$$

In Rust this is not a runtime value but a documentation-and-test contract: each public type in a core crate must have a doc comment enumerating these ten fields, and a corresponding test module that exercises at least its gate, trace and residue behavior. A type without a residue policy is not admissible into a core crate.

3 Rust Type Discipline and Identifiers

This chapter fixes the low-level conventions that every crate obeys. They exist so that correctness properties of SSK Ω (recoverability, fail-closed materialization, residue visibility) are enforced by the type system wherever possible rather than by discipline alone.

3.1 Newtype identifiers and content addressing

All identifiers are newtypes over a fixed-width content hash. This gives content-addressed, comparable, serializable IDs and prevents accidental mixing of ID kinds.

```rust
/// 32-byte content digest. The concrete hash function is supplied by a
/// `HashProvider`; the runtime never hardcodes an algorithm.
#[derive(Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
pub struct Digest([u8; 32]);

macro_rules! content_id {
    ($name:ident) => {
        #[derive(Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord,
                 serde::Serialize, serde::Deserialize)]
        pub struct $name(pub Digest);
        impl $name {
            /// Derive the id from the canonical serialization of `bytes`.
            pub fn of(h: &dyn HashProvider, bytes: &[u8]) -> Self {
    $name(h.digest(bytes)) }
        }
    };
}

content_id!(CoreId);        // canonical client core
content_id!(FacetId);       // mirror facet
content_id!(LocatorId);     // locator facet
content_id!(StitchId);      // BCIK stitch
content_id!(EdgeId);        // explicit stitch edge
content_id!(CapabilityId); // capability witness
content_id!(RunId);         // one runtime execution (run descriptor)
content_id!(TraceId);       // trace record
content_id!(ResidueId);     // residue register entry
content_id!(EpochId);       // scheduler epoch
```

Identifier rule

An identifier is derived only from the *canonical* serialization of the object it names. Two runs that produce the same canonical object produce the same id; this is the substrate for deterministic replay (Chapter 16).

3.2 Sealed traits for foundation invariants

Foundation traits are *sealed*: downstream crates may depend on them and call them but may not implement them. This prevents an adapter crate (a VPN, proxy or relay backend) from forging a core object or claiming to be a gate.

```rust
mod sealed { pub trait Sealed {} }
```

```rust
/// Only foundation types implement `CoreObject`; downstream crates cannot.
pub trait CoreObject: sealed::Sealed {
    fn object_kind(&self) → ObjectKind;
    fn residue_policy(&self) → ResiduePolicy;
}
```

3.3 Typestate for the connection lifecycle

The connection lifecycle is encoded as a typestate machine so that an illegal transition is a compile error, not a runtime bug. Phantom markers carry the state; each transition consumes `self` and returns the next state or a typed rejection.

```rust
use core::marker::PhantomData;

pub struct Intent;          pub struct Canonical;
pub struct Split;           pub struct Placed;
pub struct GatePassed;      pub struct Materialized;

pub struct Connection<S> { inner: ConnectionInner, _s: PhantomData<S> }

impl Connection<Intent> {
    pub fn canonicalize(self, c: &CoreCanonicalizer)
        → Result<Connection<Canonical>, FaultKind> { /* ... */ }
}
impl Connection<Canonical> {
    pub fn birefract(self, p: &Policy)
        → Result<Connection<Split>, FaultKind> { /* ... */ }
}
impl Connection<Split> {
    pub fn place(self, cube: &Hypercube, theta: &PhaseVector)
        → Result<Connection<Placed>, FaultKind> { /* ... */ }
}
impl Connection<Placed> {
    /// Only a `Placed` connection can be gated; only a `GatePassed`
    /// connection can be materialized. Enforced at compile time.
    pub fn gate(self, gv: &GateVector, p: &Policy)
        → GateOutcome<Connection<GatePassed>> { /* ... */ }
}
impl Connection<GatePassed> {
    pub fn materialize(self, tensor: &mut StitchTensor)
        → Result<Connection<Materialized>, FaultKind> { /* ... */ }
}
```

Typestate guarantee

`StitchTensor::materialize` is reachable only from `Connection<GatePassed>`. There is no public path from `Placed` or `Split` to a tensor mutation. This makes Invariant "fail-closed materialization" a property of the API shape, not merely of the runtime logic.

3.4 Secrets and zeroization

Ephemeral key material, capability secrets and session keys are wrapped so they are zeroed on drop and never printed.

```rust
use zeroize::{Zeroize, ZeroizeOnDrop};

#[derive(ZeroizeOnDrop)]
pub struct SecretBytes(Vec<u8>);

impl core::fmt::Debug for SecretBytes {
    fn fmt(&self, f: &mut core::fmt::Formatter<'_>) → core::fmt::Result {
        f.write_str("SecretBytes(<redacted>)")
    }
}
```

3.5 Error and result conventions

Every fallible foundation operation returns `Result<T, FaultKind>` where `FaultKind` is the crate-local slice of the global `Fault` taxonomy (Chapter 17). Errors are built with `thiserror`; panics are reserved for provably-unreachable states and are gated behind `debug_assert!` where possible. `unsafe` is forbidden by crate attribute (`#![forbid(unsafe_code)]`).

4 Core Domain Model

The domain model is the timeless, I/O-free heart of the runtime. It lives in `ssk-types` and `ssk-core` and is `no_std`-friendly (allocation allowed, no OS). Every type here is `serde`-serializable so that any object can be traced, hashed and replayed.

4.1 Core node and mirror facets

```rust
/// The canonical client identity. Never exposed as a public locator.
#[derive(Clone, Serialize, Deserialize)]
pub struct CoreNode {
    pub id: CoreId,
    pub canonical: CanonicalForm,   // xc = Can(xc) must hold
    pub signature: CoreSignature,   // over canonical form
    pub policy_root: PolicyRootRef,
    pub replay_root: ReplayRoot,
    pub audit_root: AuditRoot,
    pub closure: ClosureState,
}

/// A projection of exactly one CoreNode. Ephemeral, identity-side only.
#[derive(Clone, Serialize, Deserialize)]
pub struct MirrorFacet {
    pub id: FacetId,
    pub core: CoreId,               // the single core this facet belongs to
    pub phase: PhaseVector,         // theta in T^n
    pub orientation: Orientation,
    pub sector: Sector,
    pub key_ctx: KeyContextRef,     // ephemeral, zeroized material by ref
    pub session: SessionId,
    pub policy: PolicyId,
    pub trace: TraceId,
    pub boundary: BoundaryRef,
    pub residue: ResidueRef,
}
```

Invariant — projection recoverability

For every valid `MirrorFacet` there exists a replayable relation $\mathsf{RecoverCore}(f, \mathsf{trace}, \mathsf{replay}) = \mathsf{core.id}$. The runtime retains this relation internally for audit and focus; external observers do not receive it. In Rust: `MirrorOrbit::recover(&self, facet: &MirrorFacet) -> Result<CoreId, FaultKind>` must be total over facets produced by the same orbit, and property-tested (Chapter 18).

```rust
/// Deterministic set of facets derived from one core at a phase state.
pub struct MirrorOrbit<'c> { core: &'c CoreNode, seed: OrbitSeed }

impl<'c> MirrorOrbit<'c> {
    pub fn derive(&self, phase: &PhaseVector, sector: Sector) -> MirrorFacet { /* ... */ }
    pub fn recover(&self, facet: &MirrorFacet) -> Result<CoreId, FaultKind> { /* ... */ }
```

```
}
```

4.2 Birefringent split: identity ray and locator ray

A connection intent is never executed directly. It is split into two typed rays. The split is a pure function; it performs no I/O.

```
pub fn birefract(c: &ConnectionIntent, core: &CoreNode, p: &Policy)
    → Result<(IdentityRay, LocatorRay), FaultKind>;

/// "What the core intends to do."
#[derive(Clone, Serialize, Deserialize)]
pub struct IdentityRay {
    pub core_ref: CoreId,
    pub session_intent: SessionIntent,
    pub key_context: KeyContextRef,
    pub app_policy: PolicyId,
    pub replay_need: ReplayNeed,
    pub residue_policy: ResiduePolicy,
}

/// "How the connection may appear externally."
#[derive(Clone, Serialize, Deserialize)]
pub struct LocatorRay {
    pub allowed_locator_types: LocatorTypeSet,
    pub route_class: RouteClass,
    pub transport_class: TransportClass,
    pub timing_class: TimingClass,
    pub risk_budget: RiskBudget,
    pub cost_budget: CostBudget,
}
```

Invariant — plane separation

For every connection event e, $\pi_{\mathrm{id}}(e) \neq \pi_{\mathrm{loc}}(e)$ unless the runtime is explicitly in direct-IP baseline mode; even then the equality is recorded as a *high-linkability locator condition*, never as identity equivalence. In Rust the two rays are distinct types with no `From`/`Into` conversion between them, so they cannot be accidentally unified.

4.3 Locator facets

Locator facets are the only place where external transport appears in the type system. They are a closed enum of egress classes plus the observable, accountable metadata of each.

```
#[derive(Clone, Serialize, Deserialize)]
pub enum LocatorKind {
    DirectIp, VpnExit, ProxyExit, Relay, Gateway,
    MultiHopPath, LocalOverlay, ControlledEgress, TestHarness,
}

#[derive(Clone, Serialize, Deserialize)]
pub struct LocatorFacet {
    pub id: LocatorId,
    pub kind: LocatorKind,
    pub endpoint: EndpointRef,      // opaque handle, resolved by an adapter
```

```
    pub route: RouteClass,
    pub transport: TransportClass,
    pub epoch: EpochId,
    pub ttl: Ttl,
    pub trust: TrustClass,          // Untrusted | Weak | Managed | Owned
    pub cost: CostClass,
    pub risk: RiskClass,
    pub health: HealthStatus,       // Unknown | Live | Degraded | Down
    pub trace: TraceId,
}
```

Axiom — locator reality condition

`LocatorPool::claims_external_plurality()` returns `true` only if the pool contains at least two distinct, live locator facets whose observable surfaces differ. A pool with a single `DirectIp` facet returns `false` and the runtime records a high-linkability baseline. This axiom is enforced at policy-load time and re-checked per epoch.

4.4 Protocol fibers

A protocol fiber is a typed transport operation carrying a valid payload or control frame. It is the boundary between the runtime and any concrete network I/O, and is always pluggable.

```
pub trait ProtocolFiber {
    type Frame;
    fn fiber_type(&self) → FiberType;
    /// Must preserve fiber invariants or return a typed FiberResidue.
    fn transform(&self, input: Self::Frame) → Result<Self::Frame, FaultKind>;
}
```

5 Cryptographic and Capability Layer

Non-goal (normative)

SSK Ω contains **no self-designed cryptography**. It defines only *provider traits*. Concrete implementations must be backed by established, audited crates (for example `ed25519-dalek`, `x25519-dalek`, `chacha20poly1305`, `sha2`/`blake3`, or an HSM/KMS binding). The runtime treats these as opaque and never rolls its own primitive.

5.1 Provider traits

All cryptographic capability enters the runtime through five sealed provider traits. They are object-safe so they can be held as `Arc<dyn ...>` and swapped per deployment.

```rust
pub trait HashProvider: Send + Sync {
    fn digest(&self, bytes: &[u8]) → Digest;
    fn algorithm(&self) → HashAlg;   // recorded in trace for replay
}

pub trait SignatureProvider: Send + Sync {
    fn verify(&self, key: &PublicKey, msg: &[u8], sig: &Signature) → Result<(),
    CryptoError>;
    fn sign(&self, key: &SigningKeyRef, msg: &[u8]) → Result<Signature, CryptoError>;
    fn scheme(&self) → SigScheme;
}

pub trait KeyAgreementProvider: Send + Sync {
    fn agree(&self, local: &PrivateKeyRef, remote: &PublicKey)
        → Result<SharedSecret, CryptoError>;
    fn scheme(&self) → KexScheme;
}

pub trait AeadProvider: Send + Sync {
    fn seal(&self, key: &AeadKeyRef, nonce: &Nonce, aad: &[u8], pt: &[u8])
        → Result<Vec<u8>, CryptoError>;
    fn open(&self, key: &AeadKeyRef, nonce: &Nonce, aad: &[u8], ct: &[u8])
        → Result<Vec<u8>, CryptoError>;
    fn scheme(&self) → AeadScheme;
}

/// Attests that an action/stitch is admissible under policy WITHOUT
/// exposing the core identity as a locator address.
pub trait CapabilityWitnessProvider: Send + Sync {
    fn issue(&self, claim: &CapabilityClaim, subject: &SubjectClass)
        → Result<CapabilityWitness, CryptoError>;
    fn verify(&self, w: &CapabilityWitness, ctx: &VerifyCtx)
        → Result<CapabilityVerdict, CryptoError>;
    fn kind(&self) → WitnessKind;   // Signature | Token | ZkStyle | PolicyCert
}
```

5.2 Capability witness

A capability witness is scoped evidence that a facet, action, locator use or stitch event is admissible. It is *not* identity and it proves only its declared claim.

```rust
#[derive(Clone, Serialize, Deserialize)]
pub struct CapabilityWitness {
    pub id: CapabilityId,
    pub claim: CapabilityClaim,     // e.g. "may use locator class Relay in epoch E"
    pub scope: Scope,               // action + facet-class binding
    pub issuer: IssuerRef,
    pub subject_class: SubjectClass,
    pub epoch: EpochId,
    pub expiry: Ttl,
    pub proof: ProofBytes,          // opaque; validated by the provider
    pub trace: TraceId,
}
```

Axiom — proof scope

A capability witness proves only its declared `claim`. It does not prove payload truth, locator safety, absence of correlation, or validity outside its `scope`/`epoch`. The capability gate (Chapter 11) accepts a witness only if it is valid, unexpired, scoped to the requested action, bound to the facet class, and replayable inside the current run descriptor.

Privacy property of witnesses

The intended privacy gain is that admission is decided by *scoped capability* rather than by revealing the core identity. A relay or gateway learns "this request carries a valid capability for locator class X in epoch E", not "this is core C". Whether a ZK-style provider is used is a deployment choice; the runtime only requires that the witness verification does not require disclosing `CoreId` to the locator side.

6 Locator and Relay Model

This chapter defines locators strictly as *technical egress/transport resources* with typed accounting. It contains no operational concealment, evasion, or circumvention procedure. An adapter is responsible for actually reaching an endpoint; the runtime only types, budgets, audits and schedules its use.

6.1 Locator classes and metadata

The classification metadata attached to every `LocatorFacet` is closed and small, so it can be reasoned about and tested exhaustively.

Field	Enum	Meaning
`TrustClass`	Untrusted, Weak, Managed, Owned	Operator's trust in the egress surface
`CostClass`	Free, Low, Metered, High	Budgeting for locator selection
`RiskClass`	Low, Medium, High, Critical	Linkability/exposure risk estimate
`HealthStatus`	Unknown, Live, Degraded, Down	Liveness/route state from probes
`Epoch/Ttl`	—	Rotation window and lifetime

6.2 Locator pool and adapter interface

The `LocatorPool` owns available locators and their health; concrete backends implement a `LocatorAdapter`. The adapter surface is deliberately minimal: resolve, probe, and release. It never receives the `CoreId`.

```
pub trait LocatorAdapter: Send + Sync {
    fn kind(&self) → LocatorKind;
    /// Resolve an opaque endpoint handle for use; no CoreId is passed in.
    fn resolve(&self, req: &EgressRequest) → Result<EndpointRef, FaultKind>;
    fn probe(&self, ep: &EndpointRef) → HealthStatus;
    fn release(&self, ep: EndpointRef);
}

pub trait LocatorPool: Send + Sync {
    fn claims_external_plurality(&self) → bool;
    fn candidates(&self, ray: &LocatorRay, epoch: EpochId) → Vec<LocatorFacet>;
    fn health(&self, id: LocatorId) → HealthStatus;
    fn account(&mut self, id: LocatorId, use_record: LocatorUse);
}
```

6.3 Explicit stitch edges and the funnel-DAG

When multi-hop or overlay routing is available, it is represented as an explicit DAG of typed edges. Edges are never hidden carriers; each carries id, lifetime, policy reason, typed transform, boundary behavior and trace.

```rust
#[derive(Clone, Serialize, Deserialize)]
pub struct StitchEdge {
    pub id: EdgeId,
    pub src: LocatorId,
    pub dst: LocatorId,
    pub map: EdgeTransform,        // Routing | Framing | TransportAdapt |
    RelayHandoff
    pub lifetime: Ttl,
    pub policy_reason: PolicyReason,
    pub capability: Option<CapabilityId>,
    pub boundary: BoundaryRef,
    pub trace: TraceId,
}

#[derive(Clone, Serialize, Deserialize)]
pub struct LocatorGraph {
    pub nodes: Vec<LocatorFacet>,
    pub edges: Vec<StitchEdge>,     // ordinary transitions
    pub hyperedges: Vec<HyperEdge>, // multi-dependency transitions
}
```

Invariant — edge explicitness

Every `StitchEdge` has an id, lifetime, policy reason, typed transform, boundary behavior and trace. An edge missing any of these fields fails deserialization validation and cannot enter the graph. `EdgeTransform` is a closed enum; there is no "opaque tunnel" variant.

6.4 Path selection

Given a locator graph, the scheduler and gates jointly select a path. Selection may return a direct locator, a single egress, a relay, or an ordered multi-hop path. The selection function is pure over its inputs so it is replayable.

```rust
pub fn select_path(theta: &PhaseVector, g: &LocatorGraph,
                   cube: &Hypercube, p: &Policy) → Result<LocatorPath, FaultKind>;
```

7 Hypertoroidal Scheduler

The hypertorus T^n is normalized into concrete integer/fixed-point numeric structures. It is a deterministic phase scheduler; wall-clock time enters only through the runtime driver, never through the domain model. This is what makes replay possible.

7.1 Phase representation

Phase coordinates are fixed-point integers in $[0, M)$ per axis, so that arithmetic is exact and reproducible across platforms (no float nondeterminism).

```rust
pub const PHASE_MODULUS: u32 = 1 << 20; // fixed-point circle resolution

#[derive(Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub struct Phase(pub u32); // in [0, PHASE_MODULUS)

/// One coordinate per scheduler axis. n = 7 by default.
#[derive(Clone, Serialize, Deserialize)]
pub struct PhaseVector {
    pub identity: Phase, pub locator: Phase, pub protocol: Phase,
    pub timing: Phase,   pub risk: Phase,    pub replay: Phase,
    pub focus: Phase,
}

#[derive(Clone, Serialize, Deserialize)]
pub struct FrequencyVector { /* per-axis increment per tick */ pub incr: [u32; 7] }

#[derive(Clone, Copy, Serialize, Deserialize)]
pub struct Tick(pub u64);
#[derive(Clone, Copy, Serialize, Deserialize)]
pub struct Window { pub epsilon: u32 } // visibility half-width
```

7.2 Deterministic and seeded modes

The scheduler has two modes. Both are deterministic given their inputs; the difference is only whether a seed perturbs increments.

```rust
pub enum SchedulerMode {
    /// Pure: phase advances by a fixed FrequencyVector. Fully reproducible.
    Deterministic,
    /// Seeded: a recorded seed drives a deterministic PRNG that jitters
    /// increments; the seed is stored in the RunDescriptor so replay is exact.
    Seeded(SchedulerSeed),
}

pub struct HypertoroidalScheduler {
    theta: PhaseVector,
    freq: FrequencyVector,
    mode: SchedulerMode,
    epoch: EpochId,
}

impl HypertoroidalScheduler {
    pub fn tick(&mut self) -> Tick { /* advance each axis mod PHASE_MODULUS */ }
```

```
    pub fn phase(&self) → &PhaseVector { &self.theta }
    /// Visibility predicate: R_eps(x, theta) = 1{ d_T(theta, psi(x)) < eps_x }.
    pub fn visible(&self, target: &PhaseVector, w: Window) → bool { /* toroidal dist
    */ }
}
```

Definition — toroidal distance

For phases a, b on one axis, $d(a,b) = \min(|a-b|, M - |a-b|)$ with $M =$`PHASE_MODULUS`. The vector distance d_T is the max (or configured L_p) over axes. `visible` is a *necessary* precondition for candidate formation, never a sufficient condition for materialization (see Proposition "phase activation is necessary but not sufficient").

Replay requirement

The tuple (θ_0, `freq`, `mode`, `epoch`, tick count) fully determines every phase decision of a run. It is embedded in the `RunDescriptor`. Replaying a run with the same descriptor must reproduce identical phase points, path selections and visibility decisions bit-for-bit.

8 Projective Connection Hypercube

The Projective Connection Hypercube (PCH) is the feasibility workbody. A stitch candidate is a point (or path) in the product space of connection axes, and it must project consistently onto every required axis. In Rust the PCH is a set of pure projection predicates plus a weight function; it holds no I/O and is trivially unit-testable.

8.1 Axes

$$H_{\text{PCH}} = A_{\text{id}} \times A_{\text{loc}} \times A_{\text{proto}} \times A_{\text{time}} \times A_{\text{trust}} \times A_{\text{risk}} \times A_{\text{cap}} \times A_{\text{replay}} \times A_{\text{boundary}}.$$

```rust
#[derive(Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum Axis {
    Identity, Locator, Protocol, Timing, Trust, Risk, Capability, Replay, Boundary,
}

/// The projection of a candidate onto one axis.
pub enum Projection {
    Ok(AxisWeight),
    Undefined,
    Inconsistent(Reason),
    OverBudget(Reason),   // → Hold
    ReplayGap(Reason),
}

pub struct ConnectionCell {
    pub addr: CellAddr,
    pub weight: CellWeight,    // (mass, coherence, phase_ready, risk, residue,
    trace, gate)
    pub constraints: Vec<Constraint>,
    pub gate: GateStatus,
    pub boundary: BoundaryRef,
    pub trace: TraceId,
    pub residue: ResidueRef,
}
```

8.2 Projection check

```rust
pub trait AxisProjector { fn project(&self, sc: &StitchCandidate) → Projection; }

pub struct Hypercube { projectors: [Box<dyn AxisProjector>; 9] }

impl Hypercube {
    pub fn check(&self, sc: &StitchCandidate) → PchOutcome {
        for (axis, p) in self.projectors.iter().enumerate() {
            match p.project(sc) {
                Projection::Ok(_)              ⇒ continue,
                Projection::Undefined          ⇒ return PchOutcome::fail(axis,
    "undefined"),
                Projection::Inconsistent(r)    ⇒ return PchOutcome::fail(axis, r),
                Projection::ReplayGap(r)       ⇒ return PchOutcome::fail(axis, r),
                Projection::OverBudget(r)      ⇒ return PchOutcome::hold(axis, r),
```

```
            }
        }
        PchOutcome::Pass
    }
}
```

Invariant — projection consistency

$$\exists a \in \mathsf{Axes} : \mathsf{Invalid}(\pi_a(sc)) \Rightarrow \mathsf{Pass}_{\mathrm{PCH}}(sc) = 0.$$

A candidate may not pass PCH if any required axis projection is undefined, contradictory, over budget, or non-replayable. `OverBudget` yields *Hold* (diagnosable, non-persistent); all other failures yield *Fail* with a typed residue.

9 BCIK and Boundary Model

The Boundary-Closed Invariant Kernel (BCIK) makes an identity/locator crossing a *closure object*, not merely a routing decision. Every crossing must produce a seam witness, a boundary record, and either full closure or a visible residue. This chapter fixes the Rust surface of the kernel and the exhaustive residue taxonomy.

9.1 Crossing kernel

```rust
pub struct Bcik<'p> {
    null_ref: NullRef,              // Z: external null reference (not a network state)
    boundary: BoundaryOperator,     // partial-d
    seam: SeamInterface,            // Sigma
    residue: ResidueRegister,       // R_res
    provider: &'p dyn HashProvider,
}

/// A boundary-closed identity/locator/fiber binding.
#[derive(Clone, Serialize, Deserialize)]
pub struct BcikStitch {
    pub id: StitchId,
    pub facet: FacetId,
    pub locator: LocatorId,
    pub fiber: FiberType,
    pub phase: PhaseVector,
    pub capability: Option<CapabilityId>,
    pub seam: SeamWitness,
    pub boundary: BoundaryRecord,
    pub residues: Vec<ResidueId>,
    pub gate_report: Option<GateReportId>,
    pub trace: TraceId,
    pub replay: ReplayRef,
}

impl<'p> Bcik<'p> {
    /// Cross the seam. Either returns a closed stitch OR a boundary residue;
    /// never a silent success and never a silent drop.
    pub fn cross(&mut self, facet: &MirrorFacet, loc: &LocatorFacet,
                 fiber: FiberType, p: &Policy) -> CrossOutcome;
}

pub enum CrossOutcome {
    Closed(BcikStitch),
    Residue(BoundaryResidue),   // typed, visible, recorded
}
```

Invariant — no silent crossing

No identity/locator crossing is valid without a BCIK witness. A crossing lacking seam, boundary, gate, trace and replay is not a stitch; it is a `BoundaryResidue`. In Rust, `cross` returns `CrossOutcome`, which has no "unit success" variant: the caller is forced to handle either a closed stitch or a residue.

Definition — stitch closure

$$\mathsf{Closed}(s) \iff P^{-}s = 0, \qquad \mathsf{Closed}^{\epsilon}(s) \iff \|P^{-}s\| \le \epsilon_{\text{stitch}}.$$

P^{-} is the residual projector: the part of the crossing not absorbed into a clean binding. The tolerance ϵ_{stitch} is policy-defined; a nonzero residual within tolerance is still recorded, never discarded.

9.2 Residue taxonomy

Residues are the visible currency of "no silent loss". The taxonomy is a closed enum so that Faultscan and the audit layer can reason about it exhaustively.

```rust
#[derive(Clone, Serialize, Deserialize)]
pub enum ResidueKind {
    Identity,      // unresolved core/facet linkage
    Locator,       // locator leak, trust failure, exhaustion, route ambiguity
    Timing,        // residual timing correlation after shaping
    Protocol,      // fiber invariant not preserved
    Capability,    // proof/scope/epoch failure
    Replay,        // run descriptor insufficient / nondeterminism
    Correlation,   // cross-surface correlation risk
    Focus,         // ambiguous reanalysis reconstruction
    Boundary,      // unclosed seam
}

#[derive(Clone, Serialize, Deserialize)]
pub struct BoundaryResidue {
    pub id: ResidueId,
    pub kind: ResidueKind,
    pub severity: Severity,
    pub origin: TraceId,
    pub detail: ResidueDetail,
    pub open_obligation: bool,    // must policy quarantine/hold until resolved?
}

pub struct ResidueRegister { entries: Vec<BoundaryResidue> }
impl ResidueRegister {
    pub fn emit(&mut self, r: BoundaryResidue) -> ResidueId { /* append + hash-chain
    */ }
    pub fn open_obligations(&self) -> impl Iterator<Item = &BoundaryResidue> { /* ...
    */ }
}
```

Contract — residue completeness

If a valid crossing produces an unresolved identity, locator, boundary, capability, focus or replay component, that component *must* appear in the residue register. A missing residue field for an unresolved component is itself a `Critical` fault ("silent residue attempt"), which fails closed.

10 Gabriel Constraint Cells

Gabriel cells are local, advisory constraint evaluators over the seam and PCH cells. They are *not* authorities and hold *no persistence rights*: a cell may propose, hold, refine, reject or quarantine a local candidate, but only the global gate vector can authorize a tensor mutation.

```rust
pub enum CellRole {
    Identity, Locator, Protocol, Timing, Risk, Capability, Boundary, Replay, Focus,
}

pub enum CellVerdict { Propose, Hold(Reason), Refine(Hint), Reject(Reason),
    Quarantine(Reason) }

pub trait ConstraintCell: Send + Sync {
    fn role(&self) → CellRole;
    /// Local, side-effect-free evaluation over a bounded local view.
    fn evaluate(&self, cand: &StitchCandidate, view: &LocalView, p: &Policy) →
    CellVerdict;
}

/// The Gabriel film: all active cells at a phase state.
pub struct GabrielFilm { cells: Vec<Box<dyn ConstraintCell>> }
impl GabrielFilm {
    pub fn evaluate_all(&self, cand: &StitchCandidate, p: &Policy) →
    Vec<CellVerdict>;
}
```

Contract — cellular non-persistence

A `ConstraintCell` receives only a bounded `LocalView` and returns a `CellVerdict`. It has no reference to `&mut StitchTensor` and no interior mutability over shared runtime state. This is enforced by the trait signature: `evaluate` takes `&self` and shared references only.

11 Gate System and Gate Vector

The gate vector is the security and correctness surface. It is an ordered conjunction of gates, each of which is a pure predicate producing a witness, a failure mode, a residue emission, an audit record and a replay requirement. The vector is **fail-closed**: a single failed mandatory gate prevents materialization.

11.1 Gate contract

```rust
pub struct GateWitness { pub gate: GateId, pub evidence: EvidenceRef }

pub enum GateVerdict {
    Pass(GateWitness),
    Hold(ResidueId),
    Refine(ResidueId),
    Reject(ResidueId),
    Quarantine(ResidueId),
}

/// Every gate: input type, output witness, failure mode, residue, audit, replay.
pub trait Gate: Send + Sync {
    fn id(&self) → GateId;
    fn required(&self) → bool;                  // mandatory gates cannot be skipped
    fn residue_on_fail(&self) → ResidueKind;    // typed residue emitted on failure
    fn evaluate(&self, sc: &StitchCandidate, ctx: &GateCtx) → GateVerdict;
}
```

11.2 The twelve gates

Gate	Checks	Residue on fail
`CoreGate`	canonicalization, core signature, policy root	`Identity`
`MirrorGate`	facet in core orbit, sector valid, recoverable	`Identity`
`LocatorGate`	locator exists, type permitted, trust/cost/risk bounds	`Locator`
`BcikGate`	seam, boundary, residual projector, no silent crossing	`Boundary`
`HypercubeGate`	all PCH projections defined and compatible	`Boundary`
`CapabilityGate`	witness valid, scoped, unexpired, replayable	`Capability`
`FiberGate`	fiber typed, transform invariants preserved	`Protocol`
`PrivacyGate`	linkability within budget; metadata budget respected	`Correlation`
`BoundaryGate`	residue register consistent; no missing residue field	`Boundary`
`FocusGate`	responses can refocus to canonical core class	`Focus`
`ReplayGate`	deterministic replay under run descriptor	`Replay`
`DissolutionGate`	TTL/epoch fields present and finalizable	`Timing`

Gate vector semantics

$$\mathsf{Pass}_{\Omega} = \bigwedge_{G \in \mathcal{G}} \mathsf{Pass}_{G},$$

$$\mathcal{G} = \{\text{Core, Mirror, Locator, BCIK, PCH, Capability,}\\ \text{Fiber, Privacy, Boundary, Focus, Replay, Dissolution}\}.$$

Evaluation is ordered (cheap, structural gates first). On the first `Quarantine` the vector returns `Quarantine`; else on the first `Refine`, `Refine`; else on any failure, `Reject`; only an all-pass yields `Pass`. A "positive-only" gate profile (one that cannot fail) is invalid and rejected at config load.

```rust
pub struct GateVector { gates: Vec<Box<dyn Gate>> } // ordered

impl GateVector {
    pub fn evaluate(&self, sc: &StitchCandidate, ctx: &GateCtx) → GateReport {
        let mut witnesses = Vec::new();
        let (mut refine, mut quarantine, mut reject) = (None, None, None);
        for g in &self.gates {
            match g.evaluate(sc, ctx) {
                GateVerdict::Pass(w)      ⇒ witnesses.push(w),
```

```
                GateVerdict::Hold(r)
                | GateVerdict::Refine(r)  ⇒ refine.get_or_insert(r),
                GateVerdict::Quarantine(r)⇒ { quarantine.get_or_insert(r); },
                GateVerdict::Reject(r)    ⇒ { reject.get_or_insert(r); },
            };
        }
        GateReport::resolve(witnesses, quarantine, refine, reject)
    }
}
```

11.3 Gate profile

```
#[derive(Clone, Serialize, Deserialize)]
pub struct GateProfile {
    pub order: Vec<GateId>,
    pub thresholds: Thresholds,
    pub required_evidence: Vec<EvidenceKind>,
    pub failure_policy: FailurePolicy,   // must be NoMaterialization
    pub hold_policy: HoldPolicy,         // diagnosable, non-persistent
    pub pass_policy: PassPolicy,
}
```

12 Persistent Stitch Tensor

The Persistent Stitch Tensor is where accepted stitches materialize. It is append-structured and hash-chained for tamper-evident audit, with an in-memory default backend and a pluggable `StorageBackend`. A ledger is *optional*; there is no blockchain requirement.

12.1 Structure

```rust
#[derive(Clone, Serialize, Deserialize)]
pub struct StitchTensorEntry {
    pub id: StitchId,
    pub stitch: BcikStitch,
    pub gate_report: GateReportId,
    pub accepted_at: Tick,
    pub epoch: EpochId,
    pub ttl: Ttl,
    pub prev_hash: Digest,     // hash-chain link
    pub entry_hash: Digest,    // = H(prev_hash || canonical(self without
    entry_hash))
}

pub struct StitchTensor {
    graph: StitchGraph,             // G_s: accepted stitch graph
    entries: Vec<StitchTensorEntry>,
    head: Digest,                   // current chain head
    backend: Box<dyn StorageBackend>,
}
```

12.2 Update semantics

```rust
impl StitchTensor {
    /// Only reachable with gate-passed candidates (typestate, Ch. 3).
    pub fn apply_sorted(&mut self, accepted: Vec<Accepted>) → Result<Vec<StitchId>,
    FaultKind> {
        // deterministic ordering by (epoch, StitchId) for replay stability
        let mut acc = accepted; acc.sort_by_key(|a| (a.epoch, a.stitch.id));
        let mut ids = Vec::new();
        for a in acc {
            let entry = self.chain(a)?;          // computes prev/entry hash
            self.backend.append(&entry)?;
            self.graph.insert(&entry.stitch);
            self.head = entry.entry_hash;
            ids.push(entry.id);
        }
        Ok(ids)
    }
}

pub trait StorageBackend: Send + Sync {
    fn append(&mut self, e: &StitchTensorEntry) → Result<(), FaultKind>;
    fn get(&self, id: StitchId) → Option<StitchTensorEntry>;
    fn iter(&self) → Box<dyn Iterator<Item = StitchTensorEntry> + '_>;
```

```
    fn head(&self) → Digest;
}
```

Theorem — gate failure preserves the tensor

If no candidate passes the gate vector at tick t ($\mathsf{CU}_t = \emptyset$), then $\mathsf{ST}(t+1) = \mathsf{ST}(t)$. *Proof shape:* `apply_sorted` is only called on a non-empty `accepted` vector; with no accepted candidates the tensor is untouched. In the API this is guaranteed because acceptance requires a `Connection<GatePassed>` value, which cannot exist for a failed candidate.

Theorem — no persistent stitch without replay

Every tensor revision implies replay readiness: $sc \in \mathsf{ST}(t{+}1) \setminus \mathsf{ST}(t) \Rightarrow \mathsf{ReplayReady}(sc) = 1$. The `ReplayGate` is mandatory and part of the gate vector, so any accepted candidate has already produced a replay witness recorded alongside its entry.

12.3 Storage backends

Two backends ship by default; more are pluggable via feature flags.

- `MemoryBackend` (default, `no_std`+`alloc` compatible): a `Vec` with hash-chain; used for tests and the research testbed.
- `FileLedgerBackend` (feature `storage`): append-only file of length-prefixed canonical entries; verified on load by re-walking the hash chain.

13 Concave Reanalysis Focus

Concave Reanalysis Focus reconstructs a canonical core session from a set of responses bound to facets, locators and fibers. It merges responses *and* certifies that they reanalyze into the same canonical core class (or an explicitly versioned successor). It never asserts payload truth.

13.1 Types

```rust
#[derive(Clone, Serialize, Deserialize)]
pub struct Response {
    pub facet: FacetId, pub locator: LocatorId, pub fiber: FiberType,
    pub trace: TraceId, pub payload_digest: Digest, // payload itself is not stored
}

#[derive(Clone, Serialize, Deserialize)]
pub struct FocusReport {
    pub reconstructed: CoreSessionClass,
    pub residues: Vec<ResidueId>,
    pub replay: ReplayReportRef,
    pub linkability: LinkabilityReport,
    pub verdict: FocusVerdict,   // Pass | ClassMismatch | Ambiguous
}

pub struct FocusEngine<'p> { provider: &'p dyn HashProvider }
impl<'p> FocusEngine<'p> {
    pub fn reanalyze(&self, responses: &[Response], core: &CoreNode, p: &Policy) →
    FocusReport;
}
```

Focus pass condition

A focus pass requires $[\mathsf{CoreSession}']_{\mathrm{Can}} = [\mathsf{CoreSession}]_{\mathrm{Can}}$ or an explicitly allowed successor $[\mathsf{CoreSession}']_{\mathrm{Can}} \to [\mathsf{CoreSession}^{+}]_{\mathrm{Can}}$, and a passing focus-path replay. Failure modes are typed: cross-facet collision, wrong-locator return, stale capability, replay divergence, boundary leak $\Rightarrow$ `Focus` residue.

Invariant — focus does not create truth

Focus reconstructs an internal session class from response evidence. It does not prove payload truth, endpoint trustworthiness, or absence of observation. This is a documented, tested limit, not a hidden assumption.

14 Runtime Semantics and State Machine

The end-to-end runtime is a single deterministic pipeline over one connection intent. Async appears only at the protocol-fiber I/O edge and at locator probing; the decision core is synchronous and pure so it can be replayed.

14.1 Pipeline

`ConnectionIntent` → `Canonicalization` → `CoreSignature` → `BirefringentSplit` → (`IdentityRay`, `LocatorRay`) → `MirrorOrbit` → `PCH Placement` → `HypertoroidalPhase` → `GabrielEvaluation` → `BCIK Crossing` → `CapabilityCheck` → `LocatorPath` → `PCH Projection` → `GateVector` → `TensorMaterialization` → `FiberExecution` → `ResponseCapture` → `ConcaveFocus` → `LinkabilityResidue` → `ReplayVerify` → `CommitOrDissolve`.

14.2 Legitimate outcomes

Contract — commit and dissolution

A connection event ends in exactly one of three legitimate outcomes. *Silent loss is not a legitimate outcome.*

1. **Pass:** materialized tensor update, completed focus, replay and reanalysis.
2. **Hold/Refine:** no materialization; candidate remains diagnosable and non-persistent.
3. **Dissolve/Reject/Quarantine:** no materialization; residues and audit entry remain.

```
pub enum RunOutcome {
    Pass { stitches: Vec<StitchId>, focus: FocusReport, replay: ReplayReportRef },
    HoldRefine { residues: Vec<ResidueId> },
    DissolveRejectQuarantine { residues: Vec<ResidueId>, audit: AuditRef },
}

pub fn run_once(intent: ConnectionIntent, state: &mut RuntimeState,
                policy: &Policy, rd: &RunDescriptor) → RunOutcome;
```

15 Threat and Privacy Model

Privacy in SSK Ω is defined as *controlled separation and bounded linkability*, never as invisibility. This chapter states the defensive goals, the adversary classes, what the runtime can and cannot do against each, and how privacy is translated into measurable linkability metrics rather than absolute untraceability.

15.1 Observable surfaces

$$\mathsf{Surfaces} = (\mathsf{Identity}, \mathsf{Locator}, \mathsf{Session}, \mathsf{Timing}, \mathsf{Protocol}, \mathsf{Payload}, \mathsf{Capability}, \mathsf{Replay}).$$

The goal is to keep these surfaces separable and to keep cross-surface linkage bounded and visible.

15.2 Adversary classes and honest capability matrix

Adversary	**Observes**	**Runtime's honest posture**
Local observer	Host process, config, local traffic pre-egress	Limited: local host compromise defeats separation. Zeroization and capability scoping reduce spill; no claim of resistance to a privileged local attacker.
Remote server	Requests it directly receives	Sees a locator surface + capability, not `CoreId`, *iff* a non-direct locator is used and witnesses don't leak identity.
Locator provider	Egress at its own hop	Sees traffic at its hop. Trust class labels this; multi-hop reduces single-provider linkage but does not eliminate it.
Relay operator	Its own relay hop	Same as above; explicit-edge accounting makes the exposure visible and budgeted.
Partial network observer	Some links	Bounded linkage reduction depending on locator diversity and timing shaping; measured, not asserted.
Global passive observer	All links, all timing	**Out of scope.** The runtime explicitly does not claim resistance to a global passive adversary.
Compromised client module	Internal runtime state	Sealed traits, capability scoping and audit limit blast radius; a fully compromised core is not recoverable and must be reported.

Axiom — no absolute anonymity claim

No conformance class may claim perfect anonymity or global untraceability. A class may claim *bounded linkability reduction* under a declared adversary model, a declared locator-resource model, and a declared measurement procedure. Any deployment that advertises more than this is non-conforming.

15.3 Linkability as a metric

Linkability is a declared, measured function, not a constant. The framework fixes the interface, not a universal estimator.

```rust
pub trait LinkabilityEstimator: Send + Sync {
    /// Returns a bounded risk score in [0,1] under the declared adversary model.
    fn estimate(&self, obs: &ObservedSurface, ctx: &ThreatCtx) → LinkRisk;
    fn adversary_model(&self) → AdversaryModel; // recorded in the report
}

#[derive(Clone, Serialize, Deserialize)]
pub struct LinkabilityReport {
    pub score: LinkRisk,
    pub adversary_model: AdversaryModel,
    pub high_risk_components: Vec<SurfaceComponent>, // each must have a residue
    entry
}
```

Privacy pass

$$\mathsf{Pass}_{\mathrm{Privacy}}(sc) = 1 \iff \mathsf{LinkRisk}(C, \mathsf{Obs}(sc)) \leq \epsilon_{\mathrm{privacy}}$$

and every high-risk component has a visible residue entry. A favorable aggregate score never overrides a failed mandatory gate (metrics inform gates; they do not replace them).

16 Data, Audit, and Replay Model

All runtime-relevant objects are `serde`-serializable with a fixed *canonical* encoding so that content-addressed ids and deterministic replay are well-defined. This chapter fixes the persisted record types.

16.1 Canonical encoding

Canonicalization rule

The canonical encoding is a deterministic, field-ordered binary form (recommended: canonical CBOR or a fixed `bincode` config with sorted map keys). Every id is computed over this canonical form. Two structurally-equal objects must produce byte-identical canonical encodings, hence equal ids. Non-canonical encodings are accepted on input but re-canonicalized before hashing.

16.2 Persisted record types

```rust
/// Fully determines one execution; makes replay exact.
#[derive(Clone, Serialize, Deserialize)]
pub struct RunDescriptor {
    pub run_id: RunId,
    pub policy_hash: Digest,
    pub scheduler: SchedulerState,   // theta0, freq, mode, epoch
    pub crypto_algs: CryptoAlgSet,   // hash/sig/kex/aead identifiers
    pub input_digest: Digest,        // digest of the ConnectionIntent
    pub prior_tensor_head: Digest,
}

#[derive(Clone, Serialize, Deserialize)]
pub struct TraceRecord {
    pub id: TraceId,
    pub run: RunId,
    pub step: PipelineStep,          // which stage emitted this
    pub inputs: Vec<Digest>,
    pub outputs: Vec<Digest>,
    pub gate_report: Option<GateReportId>,
    pub residues: Vec<ResidueId>,
    pub prev_hash: Digest,
    pub entry_hash: Digest,
}

#[derive(Clone, Serialize, Deserialize)]
pub struct ReplayManifest {
    pub run: RunId,
    pub descriptor: RunDescriptor,
    pub trace_head: Digest,
    pub expected_outcome_digest: Digest,
}

#[derive(Clone, Serialize, Deserialize)]
pub struct GateReport {
```

```
    pub id: GateReportId,
    pub outcome: GateOutcomeKind,    // Pass | Hold | Refine | Reject | Quarantine
    pub witnesses: Vec<GateWitness>,
    pub residues: Vec<ResidueId>,
}

#[derive(Clone, Serialize, Deserialize)]
pub struct ResidueReport { pub run: RunId, pub residues: Vec<BoundaryResidue> }
```

16.3 Replay engine

```
pub trait ReplayEngine {
    /// Re-executes the decision core under a manifest and diffs every step.
    fn replay(&self, m: &ReplayManifest, state: &RuntimeState) → ReplayVerdict;
}

pub enum ReplayVerdict {
    Deterministic,                      // bit-identical trace + outcome
    Divergent { first_diff: PipelineStep, detail: DiffDetail }, // → Replay residue
}
```

Replay determinism obligation

Given identical `RunDescriptor`, identical prior tensor head, and identical input digest, the decision core must reproduce identical candidates, gate reports, tensor updates and focus reports. Any divergence is a `Replay` residue and a test failure. All nondeterminism sources (time, randomness, map iteration order, float arithmetic) are excluded from the decision core by construction: time is injected, randomness is seeded and recorded, maps use ordered iteration, and phase math is fixed-point integer.

17 Faultscan

Faultscan is the runtime's exhaustive fault taxonomy and detector. Every fault has a measurement, a severity, a response and a residue. The default posture is **fail-closed**: on any `Critical` fault, no materialization occurs.

17.1 Fault taxonomy

```
#[derive(Clone, Serialize, Deserialize)]
pub enum FaultKind {
    CoreNotCanonical, FacetNotRecoverable, LocatorUnavailable, SingleIpOverclaim,
    BcikSeamMissing, PchProjectionGap, CapabilityInvalid, PrivacyBudgetExceeded,
    TimingResidueHigh, FiberInvariantFailure, FocusAmbiguity, ReplayGap,
    ReanalysisMismatch, SilentResidueAttempt, DissolutionFailure, StorageFailure,
    ConfigInvalid, CryptoProviderError,
}

#[derive(Clone, Copy, Serialize, Deserialize)]
pub enum Severity { Info, Low, High, Critical }

pub struct FaultEvent {
    pub kind: FaultKind, pub severity: Severity,
    pub detection: DetectionSource, pub residue: Option<ResidueId>, pub trace:
    TraceId,
}
```

17.2 Fault matrix

Fault	Measurement	Severity	Response
CoreNotCanonical	$\mathsf{Can}(core) \neq core$	Critical	Reject; no materialization
FacetNotRecoverable	`recover(f)` fails	Critical	Quarantine facet
LocatorUnavailable	liveness/route probe fails	High	Try alternative or hold
SingleIpOverclaim	external claim with one locator	Critical	Fail conformance claim
BcikSeamMissing	no seam/boundary witness	Critical	Reject crossing
PchProjectionGap	undefined axis projection	High	Boundary residue
CapabilityInvalid	proof/scope/epoch failure	High	Capability residue
PrivacyBudgetExceeded	$\mathsf{LinkRisk} > \epsilon$	High	Rotate, hold or reject
TimingResidueHigh	timing correlation estimate high	High	Delay, batch, report residue
FiberInvariantFailure	transform breaks type	High	Fiber residue
FocusAmbiguity	multiple class reconstructions	High	Focus residue
ReplayGap	run descriptor insufficient	High	Replay residue
ReanalysisMismatch	reconstructed class invalid	High	Reanalysis residue
SilentResidueAttempt	residue field missing	Critical	Hard fail
DissolutionFailure	expired stitch still active	High	Force cleanup + audit

```rust
pub trait Faultscan {
    fn scan(&self, ctx: &RuntimeState) → Vec<FaultEvent>;
    fn fail_closed(&self, faults: &[FaultEvent]) → bool {
        faults.iter().any(|f| matches!(f.severity, Severity::Critical))
    }
}
```

Axiom — metrics do not replace gates

A metric can inform a gate, but no aggregate score overrides a required gate. If a mandatory gate fails, the event does not materialize even when aggregate metrics look favorable.

18 Validation and Testing

Testing is a first-class deliverable, not an afterthought. The specification fixes five test tiers, the deterministic vectors they consume, and the privacy metrics they assert.

18.1 Test tiers

Tier	Scope	Tooling
Unit	Per-type invariants, gate predicates, projections	`cargo test`
Property	Invariants over generated inputs (recoverability, fail-closed)	`proptest`
Integration	Full pipeline over fixture intents	`cargo test` + fixtures
Replay	Determinism under `RunDescriptor`	replay-diff harness
Simulation	Adversary/linkability scenarios (defensive)	`ssk-sim` crate

18.2 Deterministic test vectors

Each vector is a `(RunDescriptor, ConnectionIntent, ExpectedOutcomeDigest)` triple stored in `tests/vectors/`. A conforming build must reproduce every expected outcome digest exactly.

18.3 Property obligations

```
proptest! {
    // Every facet derived from an orbit recovers to its core.
    #[test]
    fn facet_recovers_to_core(seed in any::<u64>(), phase in phase_strategy()) {
        let core = core_fixture(seed);
        let orbit = MirrorOrbit::new(&core, seed.into());
        let f = orbit.derive(&phase, Sector::default());
        prop_assert_eq!(orbit.recover(&f).unwrap(), core.id);
    }

    // A failed gate never mutates the tensor.
    #[test]
    fn failed_gate_preserves_tensor(sc in failing_candidate_strategy()) {
        let mut t = StitchTensor::in_memory();
        let before = t.head();
        let report = gate_vector().evaluate(&sc, &ctx());
        prop_assert!(!report.is_pass());
        // no typestate path to materialize exists for a non-passing candidate
        prop_assert_eq!(t.head(), before);
    }

    // Every unresolved crossing component appears as a residue.
```

```
    #[test]
    fn no_silent_loss(x in crossing_with_unresolved_component()) {
        let out = bcik().cross(&x.facet, &x.locator, x.fiber, &policy());
        prop_assert!(matches!(out, CrossOutcome::Residue(_)) ||
    out.residues_nonempty());
    }
}
```

18.4 Privacy metric tests

Privacy tests assert bounded, declared behavior, never absolutes.

- **Linkability score:** under a declared `AdversaryModel`, the estimated score for a fixture session stays within the configured budget, or a residue is emitted.
- **Locator reuse:** stable locator association across sessions is detected and reported.
- **Timing concentration:** residual timing regularity after shaping is measured and bounded.
- **Cross-facet correlation:** features stable across facets surface as a `Correlation` residue.
- **Residue visibility:** every high-risk component in a `LinkabilityReport` has a matching entry in the residue register (a completeness test).

18.5 Defensive adversarial simulation

`ssk-sim` runs local, controlled scenarios (local overlay only) that model the adversary classes of Chapter 15 to *measure* separation and residue behavior. It contains no evasion tooling and never targets external networks; it is a measurement harness over the runtime's own decisions.

18.6 Conformance test map

Test	Acceptance criterion
Core canonicalization	Deterministic and signature-stable.
Mirror orbit recovery	Every facet recovers to a core under trace/replay.
Locator reality	External plurality claimed only with distinct live resources.
Plane separation	Identity/Locator projections stay separately typed.
BCIK seam	Every crossing carries seam, boundary, gate, trace, residue.
PCH projection	All axes project or emit typed residues.
Capability scope	Witness fails outside declared scope/epoch/action.
Hypertorus determinism	Fixed descriptor reproduces phase selection.
Gabriel non-persistence	Cells propose but never persist tensor entries.
Gate vector	One failed mandatory gate prevents materialization.
Tensor preservation	No accepted updates $\Rightarrow$ unchanged tensor.
Concave focus	Responses reconstruct to same class or allowed successor.
Linkability residue	High-risk correlations appear as visible residues.
Replay determinism	Identical input + descriptor + tensor $\Rightarrow$ identical result.
Dissolution	Expired edges removed; audit records remain.
End-to-end fail-closed	Invalid event emits residue, no persistent stitch.

19 Pure-Rust Implementation Blueprint

19.1 Hard constraints

- **No C/C++ core dependency.** The decision core compiles with pure-Rust dependencies. Optional adapters (KMS/HSM, TLS) may pull system libraries but only behind feature flags and never in `ssk-core`/`ssk-types`.
- `#![forbid(unsafe_code)]` in every crate by default.
- **Async optional and encapsulated.** The decision core is synchronous; async lives at I/O edges (`ssk-locator` adapters, `ssk-cli`) behind the `async` feature.
- **Determinism.** No wall-clock, RNG, float, or unordered iteration in the decision core.

19.2 Feature flags

Flag	Effect
`std`	Enables `std` (default). Disabling targets `no_std`+`alloc` for the core.
`async`	Enables async locator adapters and CLI driver (`tokio`).
`sim`	Builds the simulation/measurement harness.
`audit`	Enables full trace + hash-chained audit records.
`crypto`	Enables default provider bindings (dalek/ChaCha/BLAKE3).
`metrics`	Enables linkability/structural metric collection.
`storage`	Enables the file ledger backend.

19.3 Recommended patterns

- **Typestate** for the connection lifecycle (Chapter 3).
- **Sealed traits** for foundation invariants; downstream cannot forge core objects.
- **Newtype ids** over content digests; no primitive id confusion.
- `zeroize` for all secret material.
- `serde` for every persisted/traceable type with canonical encoding.
- `thiserror` for error taxonomies; `FaultKind` as the shared vocabulary.
- `proptest` for invariants; deterministic vectors for regression.
- `tracing` for instrumentation; spans keyed by `RunId`/`TraceId`.

19.4 Dependency posture

Concern	Suggested crate	Confinement
Serialization	`serde`, `ciborium`/`bincode`	core + all
Errors	`thiserror`	all
Secrets	`zeroize`	core
Hashing	`blake3`/`sha2`	behind `crypto`
Signatures	`ed25519-dalek`	behind `crypto`
Key agreement	`x25519-dalek`	behind `crypto`
AEAD	`chacha20poly1305`	behind `crypto`
Property testing	`proptest`	dev-dependency
Instrumentation	`tracing`	behind `audit`/`metrics`
Async runtime	`tokio`	behind `async`, edges only

20 Repository Blueprint

The repository is a Cargo workspace of focused crates that mirror the model separation of Chapter 1. Dependencies flow one way: `ssk-types` ← `ssk-core` ← runtime crates ← adapters/CLI. No cycle is permitted.

```
ssk-omega/
  Cargo.toml                # workspace
  crates/
    ssk-types/      # ids, newtypes, serde data model, no I/O, no_std-friendly
    ssk-core/       # CoreNode, MirrorFacet, birefract, sealed traits, object cards
    ssk-scheduler/  # hypertoroidal phase scheduler (deterministic + seeded)
    ssk-identity/   # mirror orbit, recovery, identity rays
    ssk-locator/    # locator pool, adapters, funnel-DAG, path selection (async edge)
    ssk-bcik/       # seam crossing, boundary operator, residue register
    ssk-gates/      # Gate trait, GateVector, GateProfile, twelve gates
    ssk-stitch/     # StitchCandidate derivation + acceptance
    ssk-tensor/     # persistent stitch tensor, storage backends, hash chain
    ssk-focus/      # concave reanalysis focus engine
    ssk-audit/      # trace records, canonical encoding, hash-chained audit
    ssk-replay/     # run descriptor, replay engine, deterministic diff
    ssk-faultscan/  # fault taxonomy + detectors + fail-closed policy
    ssk-privacy/    # linkability estimator interface + metrics
    ssk-crypto/     # provider traits + default bindings (feature: crypto)
    ssk-config/     # Policy model, YAML/TOML load + validation
    ssk-runtime/    # run_once pipeline, RuntimeState, orchestration
    ssk-sim/        # measurement + defensive adversary simulation (feature: sim)
    ssk-cli/        # binary: run, replay, verify-ledger, sim
  docs/             # this spec, ADRs, object-card reference
  examples/         # minimal viable runtime, research testbed
  tests/            # cross-crate integration + vectors
  benches/          # scheduler, gate vector, tensor append benchmarks
```

Crate responsibility rule

Each crate owns exactly one concern from the extraction table (Chapter 2) and exposes a small, documented public API. `ssk-core` and `ssk-types` never depend on async, I/O, or crypto backends; those enter only through traits so the decision core stays pure and replayable.

21 Configuration Model

Policy is loaded from YAML or TOML, deserialized into a validated `Policy`, and hashed into the `RunDescriptor`. Invalid or "positive-only" gate profiles are rejected at load.

```yaml
system:
  name: ScorpioSync-Kaleidoscorp-Omega
  fail_closed: true
  mode: research_testbed          # personal | enterprise | research_testbed | agent
core:
  canonicalization: deterministic
  signature_policy: core_boundary_residue
  expose_core_address: false
locator_plane:
  allowed_types: [direct_ip, vpn_exit, proxy_exit, relay, gateway, multi_hop_path,
    local_overlay]
  require_locator_reality_check: true
  single_ip_external_privacy_claim: forbidden
projective_connection_hypercube:
  axes: [identity, locator, protocol, timing, trust, risk, capability, replay,
    boundary]
  require_all_axis_projection: true
hypertorus:
  dimension: 7
  phase_axes: [identity, locator, protocol, timing, risk, replay, focus]
  mode: deterministic             # or: seeded
  deterministic_replay: true
bcik:
  require_seam: true
  require_boundary_operator: true
  closure_threshold: 0.0
  tolerated_residue_threshold: policy_defined
capabilities:
  witness_required: true
  allowed_witnesses: [signature, token, zk_style, policy_cert]
  require_scope_and_expiry: true
privacy:
  adversary_model: partial_network_observer
  linkability_threshold: policy_defined
  require_residue_report: true
gates:
  order: [core, mirror, locator, bcik, pch, capability, fiber, privacy, boundary,
    focus, replay, dissolution]
  failure_policy: no_materialization
  hold_policy: diagnosable_non_persistent
replay:
  run_descriptor_required: true
  deterministic_materialization_required: true
ledger:
  append_only_audit: true
  store_payload: false
  store_trace_hash: true
dissolution:
  ttl_required: true
  finalize_residues_on_expiry: true
```

22 Conformance Levels

Conformance is staged. Each level is a testable milestone with an explicit acceptance gate; a build may claim level k only if all tests for levels $0..k$ pass.

Level	Requirement
SSKΩ-0	Typed model: `ssk-types`/`ssk-core` compile; Core, Identity Plane, Locator Plane exist and are separately typed.
SSKΩ-1	Deterministic core + scheduler: fixed-point phase, deterministic tick, replayable phase selection.
SSKΩ-2	Mirror-facet orbit + locator pool with real typing and locator-reality checks.
SSKΩ-3	BCIK `StitchCandidate` pipeline: every crossing carries seam, boundary, gate, trace, replay.
SSKΩ-4	Gate vector + boundary residues: twelve gates, fail-closed, typed residues, PCH projection checks.
SSKΩ-5	Persistent Stitch Tensor + audit: hash-chained append, storage backends, tensor-preservation theorem tested.
SSKΩ-6	Concave Reanalysis Focus + replay: class reconstruction, replay determinism harness green.
SSKΩ-7	Locator pool + relay-fabric adapters: real egress adapters behind the async edge, health probing.
SSKΩ-8	Full simulation + conformance testbed: privacy metric tests, defensive adversary sim, all vectors.
SSKΩ-9	Production-hardened runtime profile: dissolution/TTL sweeps, ledger verification, deployment audit, documented limits.

23 Implementation Phases and Minimal Viable Runtime

23.1 Minimal Viable Runtime (MVR)

The MVR is the smallest build that exercises the whole spine end-to-end. It targets SSKΩ-4 and uses only the in-memory backend and a local-overlay locator.

- `ssk-types`, `ssk-core`: Core, facet, rays, ids.
- `ssk-scheduler`: deterministic phase.
- `ssk-identity`: orbit + recovery.
- `ssk-locator`: local-overlay pool only.
- `ssk-bcik` + `ssk-gates`: crossing + all twelve gates (some trivial-but-present).
- `ssk-tensor`: in-memory hash-chained tensor.
- `ssk-runtime` + `ssk-cli`: `run_once` and a `run` subcommand.

Acceptance: one fixture intent runs to a `Pass` outcome with a materialized stitch, a full trace, and a green replay; one invalid fixture produces a `BoundaryResidue` and no stitch.

23.2 Phase plan

Phase	Deliverable	Exit criterion
P0	Workspace + `ssk-types`/`ssk-core`	SSKΩ-0 tests green
P1	Scheduler + identity orbit	SSKΩ-1/2; recovery property test green
P2	BCIK + gates + tensor (MVR)	SSKΩ-3/4/5; fail-closed + preservation tests
P3	Focus + replay + audit	SSKΩ-6; replay determinism harness green
P4	Locator adapters + funnel-DAG	SSKΩ-7; async edge behind feature
P5	Privacy metrics + sim + fault-scan	SSKΩ-8; metric + defensive sim tests
P6	Hardening, dissolution, ledger verify	SSKΩ-9; documented limits + audit

24 Implementation Contract and Stated Limits

24.1 Conformance contract

A conforming implementation must satisfy all of the following, each backed by a test in the conformance map (Chapter 18):

1. All core objects are typed and documented with an object card.
2. Public locators are never treated as identity; the two rays are non-convertible types.
3. Mirror facets are recoverable to a canonical core under replay.
4. Locator plurality is claimed only when real, distinct, live locator resources exist.
5. Every identity/locator crossing yields a BCIK stitch or a typed boundary residue.
6. Every persistent tensor entry passed the full twelve-gate vector (fail-closed).
7. Every accepted stitch is replayable and reanalyzable.
8. Every failed candidate emits a typed non-materializing outcome (Hold/Refine/Reject/Quarantine).
9. Every residue remains visible through boundary, audit and replay.
10. Every domain adapter preserves the foundation and does not redefine it.

24.2 What the system does and does not claim

Provides	Does not provide
Typed identity/locator separation	Untraceability or perfect anonymity
Fail-closed, gate-bound materialization	Resistance to a global passive observer
Explicit, budgeted locator/relay accounting	Covert channels or evasion tooling
Bounded, measured linkability with visible residues	A universal linkability estimator
Deterministic trace, replay and reanalysis	Proof of payload truth or endpoint trust
Capability-scoped admission without exposing `CoreId`	Self-designed cryptography
Auditable dissolution/TTL and append-only ledger	Mandatory blockchain

Final formula

An SSK Ω connection event is valid only if a canonical `CoreNode` is projected into recoverable mirror facets, separated from locator resources by a birefringent identity/locator split, placed into the Projective Connection Hypercube, selected by the hypertoroidal phase scheduler, evaluated by Gabriel constraint cells, crossed through the Boundary-Closed Invariant Kernel, materialized into the persistent Stitch Tensor only after the full twelve-gate vector passes, and returned through Concave Reanalysis Focus into the same canonical Core class or an

explicitly versioned successor — with every linkability residue visible, every step traced, and the whole run deterministically replayable.

End of specification. This document fixes the interfaces and obligations of a pure-Rust SSK Ω runtime. Concrete cryptographic backends, network adapters, and linkability estimators are deployment choices bound by the provider traits and non-goals stated herein.